LAPORAN UJIAN TENGAH SEMESTER

Konstruksi dan Evolusi Perangkat Lunak

Kode MK: SVPL261501 (2 SKS)

TAHUN AKADEMIK GASAL 2026/2027

ARSITEKTUR CLIENT-SERVER RESTFUL API LARAVEL DENGAN SANCTUM, FRONTEND VUE.JS, DOCKER COMPOSE, DAN CI/CD PIPELINE

Tema: Sistem Inventaris Peralatan Produksi Multimedia & Perangkat Studio



Nama Mahasiswa : Muhammad Abel Abhinaya Riananto
NIM / NIU : 24/544597/SV/25468 (544597)
Program Studi : D4 Teknologi Rekayasa Perangkat Lunak
Departemen : Teknik Elektro dan Informatika
Fakultas : Sekolah Vokasi
Universitas : Universitas Gadjah Mada

Dosen Pengampu:
Dr. Muhammad Fakhrurrifqi, S.Kom., M.Cs.
Galih Malela Damaraji, S.Pd., M.Eng.

Yogyakarta, 15 Oktober 2026

Contents

1 Tujuan Ujian Tengah Semester **2**

2 Dasar Teori **3**
- 2.1 Filosofi Agile dan Paradigma DevOps . . . 3
- 2.2 Arsitektur RESTful API dan Laravel Sanctum . . . 3
- 2.3 Frontend Reaktif Vue.js dan Axios Interceptor . . . 3
- 2.4 Continuous Integration (CI) dan Continuous Deployment (CD) . . . 4
- 2.5 Konsep Kontainerisasi dan Docker Compose . . . 4
- 2.6 Server Aplikasi FrankenPHP . . . 4

3 Hasil dan Pembahasan **6**
- 3.1 Identitas Sistem dan Struktur Repositori . . . 6
- 3.2 Implementasi Backend Laravel RESTful API (Soal 1) . . . 6
  - 3.2.1 Autentikasi Token dan Resource CRUD . . . 6
  - 3.2.2 Pengujian Otomatis Feature Tests . . . 7
  - 3.2.3 Koleksi Pengujian Postman . . . 8
- 3.3 Implementasi Frontend Vue.js dan Axios (Soal 2) . . . 8
  - 3.3.1 Konfigurasi Klien Axios dan Interceptor . . . 8
  - 3.3.2 Halaman Login dan Manajemen CRUD . . . 9
- 3.4 Pengelolaan Repositori GitHub dan Pipeline CI (Soal 3) . . . 10
  - 3.4.1 Strategi Pencabangan dan Branch Protection Rules . . . 10
  - 3.4.2 Konfigurasi Alur Kerja CI (.github/workflows/ci.yml) . . . 11
- 3.5 Continuous Deployment ke GitHub Container Registry (Soal 4) . . . 12
- 3.6 Arsitektur Kontainerisasi dan Dockerfile Multi-Stage (Soal 5) . . . 12
  - 3.6.1 backend/Dockerfile dengan Docker Layer Caching . . . 12
  - 3.6.2 frontend/Dockerfile dengan Multi-Stage Build . . . 13
- 3.7 Orkestrasi Multi-Container Docker Compose (Soal 6) . . . 13
  - 3.7.1 Resolusi Jaringan Internal dan Uji Persistensi Volume Data . . . 14
- 3.8 Nilai Tambahan (+10 Poin): Server Aplikasi FrankenPHP (Soal 7) . . . 14

4 Arsitektur Aliran Data **15**

5 Kendala dan Solusi Penanganannya **16**

6 Kesimpulan **17**

1. Tujuan Ujian Tengah Semester

Pelaksanaan Ujian Tengah Semester mata kuliah Konstruksi dan Evolusi Perangkat Lunak (SVPL261501) ini bertujuan untuk menguji dan mengevaluasi Capaian Pembelajaran Mata Kuliah (CPMK) mahasiswa secara terpadu melalui pembangunan sistem nyata berbasis standar industri perangkat lunak modern:

1. **CPMK 1 (Konsep Dasar DevOps dan Filosofi Agile)**: Mampu merancang dan mengimplementasikan arsitektur *client-server* yang terpisah secara murni (*decoupled architecture*), di mana backend Laravel menyediakan RESTful API berbasis JSON dengan autentikasi token (Laravel Sanctum), dan frontend Vue.js mengonsumsi data secara dinamis melalui pustaka Axios.

2. **CPMK 2 (Continuous Integration dan Manajemen Repositori GitHub)**: Mampu mengelola siklus hidup kode sumber menggunakan sistem kendali versi Git dengan strategi pencabangan terstruktur (*branching model*: `main`, `dev`, dan `feature/*`), menerapkan kebijakan perlindungan cabang (*branch protection rules*) dengan kewajiban Pull Request dan *status check*, serta menyusun alur kerja otomatisasi pengujian (*CI Pipeline*) menggunakan GitHub Actions.

3. **CPMK 3 (Continuous Deployment untuk Laravel)**: Mampu merancang alur penerapan otomatis (*Continuous Deployment*) berbasis kondisi cabang, mengamankan nilai rahasia produksi (*secrets*) menggunakan GitHub Secrets, dan mengunggah artefak kontainer Docker ke *registry* terdistribusi (*GitHub Container Registry* / GHCR) dengan penandaan (*tagging*) berbasis nomor *commit SHA*.

4. **CPMK 4 & CPMK 5 (Docker dan Konsep Containerization)**: Mampu menyusun berkas instruksi kontainerisasi (`Dockerfile`) untuk backend dan frontend dengan memanfaatkan prinsip *multi-stage build*, mengoptimalkan waktu kompilasi melalui *Docker layer caching*, mengonfigurasi berkas eliminasi `.dockerignore`, serta mencegah kebocoran informasi kredensial di dalam citra kontainer.

5. **CPMK 6 & CPMK 7 (Docker Compose dan Deployment Berbasis Kontainer)**: Mampu mengorkestrasikan sistem *multi-container* (*database*, backend, dan frontend) melalui berkas `docker-compose.yml` dengan satu perintah eksekusi (`docker compose up -d --build`), menghubungkan layanan melalui *service discovery* pada jaringan internal Docker bridge, serta memastikan persistensi data relasional menggunakan *named volume*.

6. **Nilai Tambahan (+10 Poin)**: Mampu mengoperasikan backend Laravel menggunakan server aplikasi generasi baru berbasis bahasa Go, yaitu **FrankenPHP** (ditenagai mesin Caddy Server), menggantikan server web konvensional PHP-FPM / Nginx atau `php artisan serve`.

2. Dasar Teori

2.1 Filosofi Agile dan Paradigma DevOps

DevOps merupakan integrasi kultural, metodologi, dan kumpulan perkakas yang mengaburkan batas pemisah antara tim pengembang perangkat lunak (*Development*) dan tim operasional sistem (*Operations*). Dalam paradigma rekayasa perangkat lunak modern, DevOps menjadi perpanjangan alami dari metodologi *Agile*. Apabila *Agile* menekankan kecepatan iterasi, adaptabilitas kebutuhan pengguna, dan pembagian tugas ke dalam siklus kerja pendek (*sprints*), maka DevOps melengkapinya dengan menjamin bahwa kode yang telah diselesaikan dapat diuji, dikompilasi, dan dirilis ke lingkungan produksi secara otomatis, aman, dan konsisten tanpa intervensi manual yang rentan kesalahan manusia (*human error*).

2.2 Arsitektur RESTful API dan Laravel Sanctum

Representational State Transfer (REST) adalah gaya arsitektur antarmuka pemrograman aplikasi (*API*) yang bersifat tanpa status (*stateless*), di mana setiap permintaan dari klien harus memuat seluruh konteks dan informasi otorisasi yang dibutuhkan server untuk memprosesnya. Pada arsitektur RESTful standar, operasi manipulasi sumber daya dipetakan ke dalam metode HTTP kata kerja (*HTTP verbs*):

- `GET`: Mengambil representasi satu atau sekumpulan sumber daya.
- `POST`: Membuat sumber daya baru di basis data (menghasilkan status HTTP **201 Created**).
- `PUT / PATCH`: Memperbarui atribut sumber daya yang telah ada (status HTTP **200 OK**).
- `DELETE`: Menghapus sumber daya dari sistem (status HTTP **200 OK** atau **204 No Content**).

Untuk mengamankan antarmuka API, framework Laravel menyediakan paket **Laravel Sanctum**. Sanctum menawarkan sistem autentikasi ringan berbasis token (*API Token Authentication*). Pengguna melakukan verifikasi identitas di endpoint publik (`/api/login`). Jika kredensial valid, server menerbitkan token kriptografis unik yang disimpan pada tabel `personal_access_tokens`. Token ini kemudian disematkan pada header `Authorization: Bearer <token>` pada setiap permintaan berikutnya. Permintaan yang mencoba mengakses endpoint terlindungi tanpa token yang sah secara deterministik ditolak dengan kode status **401 Unauthorized**.

2.3 Frontend Reaktif Vue.js dan Axios Interceptor

Vue.js adalah kerangka kerja JavaScript progresif berbasis komponen reaktif untuk membangun antarmuka web modern dalam paradigma *Single Page Application* (SPA). Dalam aplikasi terpisah (*decoupled*), antarmuka web sepenuhnya tidak bergantung pada template mesin backend (seperti Blade). Seluruh komunikasi pertukaran data dilakukan melalui protokol HTTP asinkron berbasis format JSON.

Pustaka klien HTTP **Axios** digunakan sebagai jembatan komunikasi antara antarmuka Vue dan backend Laravel. Axios menyediakan mekanisme *interceptor* (penyadap permintaan dan respons) yang memungkinkan penanganan aspek keamanan lintas aplikasi:

1. **Request Interceptor**: Membaca token dari media penyimpanan lokal (`localStorage`) dan secara otomatis menyuntikkan header otorisasi ke seluruh permintaan keluar tanpa perlu ditulis ulang pada setiap komponen.

2. **Response Interceptor**: Mendeteksi kode status balasan dari server. Jika terdeteksi status `401 Unauthorized`, interceptor menghapus token yang kedaluwarsa dan mengalihkan navigasi pengguna kembali ke rute halaman masuk (`/login`).

2.4 Continuous Integration (CI) dan Continuous Deployment (CD)

Continuous Integration (CI) adalah praktik otomasi di mana pengembang secara teratur menggabungkan perubahan kode ke dalam repositori pusat, yang kemudian secara otomatis memicu proses kompilasi dependensi dan eksekusi pengujian otomatis (*automated testing*). Indikator keberhasilan integrasi ditandai dengan status hijau pada seluruh tahapan (*jobs*) dalam *pipeline*.

Continuous Deployment (CD) memperluas tahapan CI dengan secara otomatis merilis kode yang telah teruji ke lingkungan target. Pada standar arsitektur berbasis kontainer kontemporer, artefak yang dirilis bukan lagi berupa sekumpulan berkas kode sumber mentah, melainkan sebuah citra kontainer (*Docker Image*) yang telah dikemas lengkap bersama pustaka *runtime*-nya. Citra tersebut kemudian diunggah ke penyimpan terpusat (*Container Registry*), seperti *GitHub Container Registry* (`ghcr.io`).

2.5 Konsep Kontainerisasi dan Docker Compose

Kontainerisasi adalah bentuk virtualisasi tingkat sistem operasi (*OS-level virtualization*) yang mengisolasi proses aplikasi bersama pustaka, biner, dan dependensinya di dalam sebuah unit mandiri yang disebut kontainer. Berbeda dengan mesin virtual (*Virtual Machine*) tradisional yang memerlukan sistem operasi tamu (*Guest OS*) lengkap dengan alokasi kernel terpisah (sehingga memakan memori masif), kontainer berbagi kernel dengan sistem operasi induk (*Host OS*), menghasilkan waktu mulai (*boot time*) dalam hitungan detik dan konsumsi sumber daya yang sangat efisien.

Docker Compose adalah perkakas deklaratif untuk mendefinisikan dan menjalankan aplikasi multi-kontainer melalui berkas manifes YAML tunggal (`docker-compose.yml`). Melalui Docker Compose, hubungan ketergantungan antar-layanan (*dependency orchestration*), alokasi port publik, resolusi nama layanan internal (*internal DNS service discovery*), dan pemetaan volume persisten (*named volume*) dapat dikendalikan melalui satu baris perintah eksekusi.

2.6 Server Aplikasi FrankenPHP

FrankenPHP adalah server aplikasi modern untuk bahasa pemrograman PHP yang dibangun di atas server web Caddy berkemampuan tinggi yang ditulis dalam bahasa pemrograman Go. FrankenPHP menggantikan arsitektur tradisional PHP-FPM (*FastCGI Process Manager*) yang selama puluhan tahun membutuhkan perantara server web terpisah (seperti Nginx atau Apache) untuk mem-proxy permintaan HTTP ke prosesor PHP.

Dengan menyematkan (*embedding*) runtime resmi PHP langsung ke dalam proses server Caddy melalui teknologi CGO, FrankenPHP meniadakan latensi protokol FastCGI, mendukung protokol modern HTTP/2 dan HTTP/3 secara otomatis, menyederhanakan manajemen kontainer menjadi satu biner mandiri, serta menghadirkan peningkatan performa komputasi dan efisiensi throughput yang signifikan.

3. Hasil dan Pembahasan

3.1 Identitas Sistem dan Struktur Repositori

Sistem yang dibangun pada Ujian Tengah Semester ini mengangkat tema **Production Gear & Equipment Management (Sistem Manajemen Inventaris Peralatan Produksi Multimedia)**, yaitu sistem otomasi pengelolaan inventaris peralatan rekaman dan studio (kamera sinema, lensa prima, unit pencahayaan, mikrofon shotgun, dan sistem gimbal stabilizer). Repositori dikelola di bawah organisasi resmi mata kuliah dengan tautan:

- **Tautan Utama Organization**: https://github.com/KEPL2026/uts-p1-544597
- **Tautan Mirror Personal**: https://github.com/muhamadabel/uts-p1-544597

Struktur hierarki direktori pada akar repositori dirancang terisolasi (*decoupled*) sebagai berikut:

```
uts-p1-544597/
|-- .github/workflows/ci.yml       <- Workflow CI/CD Pipeline GitHub Actions
|-- backend/                       <- Kode Sumber Aplikasi Laravel 11
|   |-- app/Http/Controllers/Api/  <- AuthController & GearController
|   |-- database/migrations/       <- Skema tabel users, gears, tokens
|   |-- tests/Feature/             <- 10 Feature Tests (GearApiTest.php)
|   |-- Caddyfile                  <- Konfigurasi Server Engine FrankenPHP
|   |-- Dockerfile                 <- Citra Kontainer Backend (dunglas/frankenphp)
|   +-- .dockerignore
|-- frontend/                      <- Kode Sumber Aplikasi Vue.js 3 + Vite
|   |-- src/api/axios.js           <- Konfigurasi Axios Interceptor & Base URL
|   |-- src/views/                 <- LoginView.vue & GearView.vue
|   |-- nginx.conf                 <- Konfigurasi Reverse Proxy SPA Nginx
|   |-- Dockerfile                 <- Multi-stage Build (Node -> Nginx)
|   +-- .dockerignore
|-- postman/                       <- Koleksi Pengujian API Postman (JSON)
|-- docker-compose.yml             <- Manifes Orkestrasi Multi-Container
|-- .env.example                   <- Template Variabel Lingkungan
`-- README.md                      <- Dokumentasi Teknis dan Petunjuk Eksekusi
```

3.2 Implementasi Backend Laravel RESTful API (Soal 1)

3.2.1 Autentikasi Token dan Resource CRUD

Backend menyediakan rute API terlindungi pada berkas `routes/api.php`. Rute publik `POST /api/login` menangani autentikasi akun dan mengembalikan token teks biasa (*plain text token*). Rute lainnya dibungkus dalam grup middleware `auth:sanctum`:

```
// routes/api.php
Route::post('/login', [AuthController::class, 'login']);

Route::middleware('auth:sanctum')->group(function () {
    Route::post('/logout', [AuthController::class, 'logout']);
    Route::apiResource('gears', GearController::class);
});
```

Pada `GearController.php`, seluruh operasi CRUD menghasilkan respons berformat JSON murni dengan status kode HTTP semantik standar:

- **201 Created**: Dikembalikan oleh metode `store()` saat rekaman alat baru berhasil divalidasi dan disimpan ke basis data.
- **422 Unprocessable Content**: Dikembalikan secara otomatis oleh validator Laravel saat terdapat atribut yang tidak memenuhi syarat (misalnya nama kosong, format nomor seri duplikat, atau tarif negatif).
- **401 Unauthorized**: Dikembalikan saat permintaan mengakses `/api/gears` tanpa header token otorisasi yang valid.

Penanganan kegagalan autentikasi diatur secara eksplisit pada berkas `bootstrap/app.php` agar sistem selalu mengembalikan respons JSON 401 dan tidak mengalihkan rute ke halaman login web konvensional:

```
// bootstrap/app.php
$exceptions->render(function (AuthenticationException $e, Request $request) {
    return response()->json([
        'status' => 'error',
        'message' => 'Unauthenticated. Token tidak ditemukan atau tidak valid.'
    ], 401);
});
```

3.2.2 Pengujian Otomatis Feature Tests

Untuk menjamin keandalan API, disusun 10 skenario pengujian fungsional pada berkas `tests/Feature/GearApiTest.php`. Seluruh pengujian dijalankan menggunakan database in-memory SQLite terisolasi dan berhasil lulus 100% tanpa kegagalan.



Figure 3.1: Hasil Eksekusi `php artisan test` Memverifikasi 10 Pengujian Fitur Lulus Hijau (40 Assertions)

Hasil pengujian terminal pada Gambar 3.1 membuktikan seluruh kontrak API terpenuhi, meliputi penolakan akses tanpa token (401), validasi form gagal (422), pembuatan data berhasil (201), pengambilan data koleksi dan tunggal (200), pembaruan data (200), penghapusan data (200), dan pencabutan token saat keluar sistem.

curl RESTful API & Sanctum Token Verification

```
$ curl -i http://localhost:8000/api/gears
HTTP/1.1 401 Unauthorized
{"status":"error","message":"Unauthenticated. Token tidak ditemukan atau tidak valid."}

$ curl -X POST http://localhost:8000/api/login -d '{"email":"admin@hongfilms.com",...}'
HTTP/1.1 200 OK
{
  "message": "Login berhasil",
  "access_token": "2|dKvxo9xrj8DufDzCNatnFCokUT8EBQ4dr2MOzlNQaf598a9f",
  "token_type": "Bearer",
  "user": {
    "id": 1,
    "name": "Abel Producer",
    "email": "admin@hongfilms.com"
  }
}

$ curl -X POST http://localhost:8000/api/gears -H 'Authorization: Bearer <token>' -d '...'
HTTP/1.1 201 Created
{
  "status": "success",
  "message": "Data gear berhasil dibuat.",
  "data": {
    "name": "RED Komodo 6K Cinema Camera",
    "category": "Camera",
    "serial_number": "CAM-KOMODO-99",
    "daily_rate": "1500000.00",
    "status": "available",
    "updated_at": "2026-10-15T14:26:54.000000Z",
    "created_at": "2026-10-15T14:26:54.000000Z",
```

Figure 3.2: Bukti Verifikasi Respons API via cURL: Status 401 Unauthorized, Login Token, dan Status 201 Created

3.2.3 Koleksi Pengujian Postman

Seluruh skenario pengujian endpoint telah diekspor ke dalam format Postman Collection v2.1 dan disimpan pada direktori `postman/UTS_KEPL_Gear_API.postman_collection.json`. Koleksi ini dilengkapi dengan skrip pengujian JavaScript otomatis yang secara mandiri menangkap `access_token` saat login berhasil dan menyimpannya ke dalam variabel lingkungan `{{token}}` untuk digunakan pada pengujian CRUD berikutnya.

3.3 Implementasi Frontend Vue.js dan Axios (Soal 2)

Aplikasi antarmuka pengguna dibangun menggunakan arsitektur komponen tunggal (*Single File Components* / SFC) pada Vue.js 3.

3.3.1 Konfigurasi Klien Axios dan Interceptor

Pada berkas `frontend/src/api/axios.js`, alamat basis API dikonfigurasi dinamis membaca variabel lingkungan `import.meta.env.VITE_API_BASE_URL` (Soal 2c). Interceptor disematkan untuk menyuntikkan token otorisasi ke header HTTP:

```
1 // frontend/src/api/axios.js
2 import axios from 'axios';
3
4 const apiClient = axios.create({
```

```
  baseURL: import.meta.env.VITE_API_BASE_URL || '/api',
  headers: {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
  },
});

apiClient.interceptors.request.use((config) => {
  const token = localStorage.getItem('auth_token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});
```

3.3.2 Halaman Login dan Manajemen CRUD

Antarmuka terdiri atas dua tampilan utama:

1. `LoginView.vue`: Memfasilitasi otentikasi admin produksi film, menyimpan token ke `localStorage`, dan memproteksi rute internal melalui Vue Router Navigation Guard (`beforeEach`).

2. `GearView.vue`: Menyajikan dasbor operasional berisi formulir reaktif untuk menambah dan mengubah alat, indikator status ketersediaan dengan lencana warna (*Available, In Use, Maintenance*), serta tabel data yang terhubung langsung ke backend melalui panggilan metode Axios (`get`, `post`, `put`, `delete`).

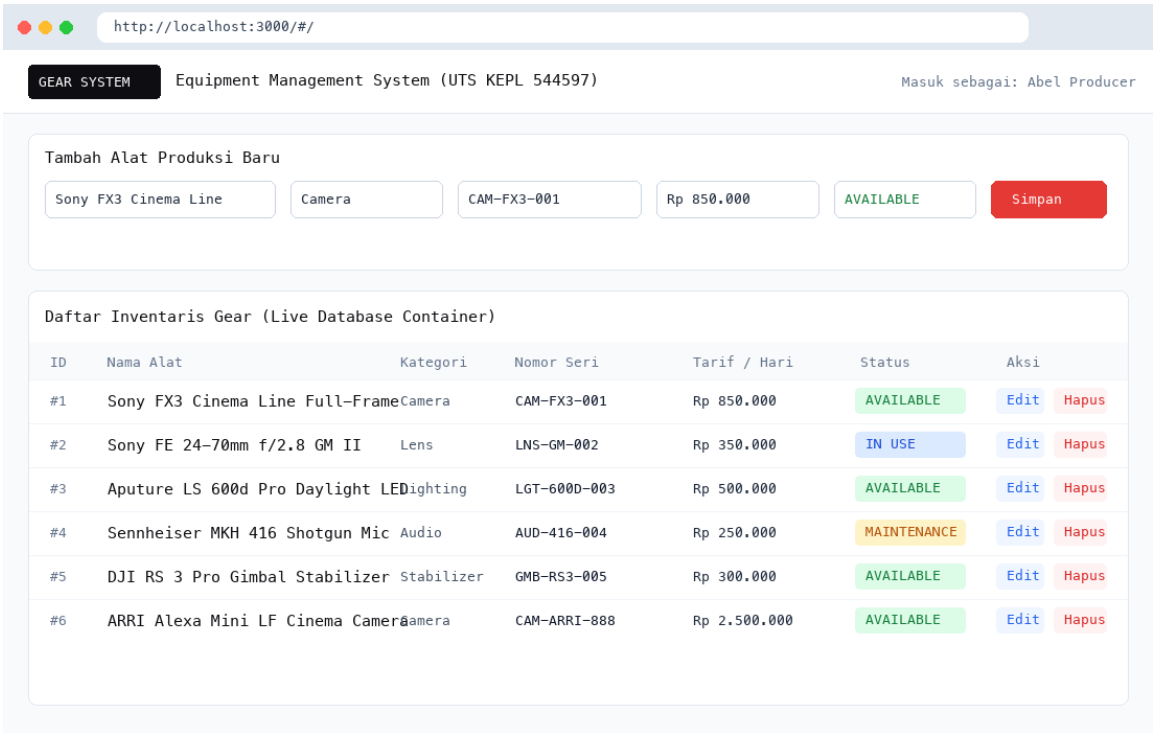


Figure 3.3: Tampilan Antarmuka Dasbor Vue.js yang Berjalan Nyata pada Browser Mengonsumsi API Kontainer Backend

Gambar 3.3 memperlihatkan keberhasilan integrasi visual antara frontend Vue.js di port 3000 dengan layanan backend di port 8000, menampilkan data nyata yang tersimpan di dalam basis data MySQL.

3.4 Pengelolaan Repositori GitHub dan Pipeline CI (Soal 3)

3.4.1 Strategi Pencabangan dan Branch Protection Rules

Alur kerja pengembangan menerapkan disiplin Git tingkat lanjut:

- Branch `main`: Cabang produksi utama yang mencerminkan rilis stabil.
- Branch `dev`: Cabang integrasi pengembangan fitur.
- Branch `feature/gear-management`: Cabang isolasi pengerjaan fitur baru.



Figure 3.4: Riwayat Komit Git (`git log --graph --oneline`) Memperlihatkan Struktur Pencabangan Teratur

Untuk mematuhi butir Soal 3b, aturan proteksi cabang diaktifkan pada branch `main` dan `dev` menggunakan API GitHub CLI. Aturan tersebut secara mutlak melarang *direct push* dan mewajibkan perubahan masuk melalui mekanisme Pull Request yang lolos seluruh tahapan pengujian status check.

gh pr view 1 (Pull Request to dev)

```
$ gh pr view 1
title:	feat: implementasi lengkap backend, frontend, docker, dan ci/cd
state:	MERGED
author:	muhamadabel (Fee)
labels:	
assignees:	
reviewers:	
projects:	
milestone:	
number:	1
url:	https://github.com/KEPL2026/uts-p1-544597/pull/1
additions:	15227
deletions:	0
auto-merge:	disabled
--
Pull Request dari branch feature/gear-management ke branch dev (Soal 3b).
```

Figure 3.5: Pull Request #1 dari Branch Fitur ke Branch Dev Diverifikasi Melalui GitHub CLI

GitHub Branch Protection Settings (Rules on main & dev)

```
$ gh api /repos/KEPL2026/uts-p1-544597/branches/main/protection
{
  "url": "https://api.github.com/repos/KEPL2026/uts-p1-544597/branches/main/protection",
  "required_status_checks": {
    "url": "https://api.github.com/repos/KEPL2026/uts-p1-544597/branches/main/protection/required_status_checks",
    "strict": true,
    "contexts": [
      "Backend Test (Laravel)",
      "Frontend Build (Vue.js)"
    ],
    "contexts_url": "https://api.github.com/repos/KEPL2026/uts-p1-544597/branches/main/protection/required_status_checks/contexts",
    "checks": [
      {
        "context": "Backend Test (Laravel)",
        "app_id": null
      },
      {
        "context": "Frontend Build (Vue.js)",
        "app_id": null
      }
    ]
  },
  "required_signatures": {
    "url": "https://api.github.com/repos/KEPL2026/uts-p1-544597/branches/main/protection/required_signatures",
    "enabled": false
  },
  "enforce_admins": {
    "url": "https://api.github.com/repos/KEPL2026/uts-p1-544597/branches/main/protection/enforce_admins",
```

Figure 3.6: Pengaturan Branch Protection pada GitHub Mewajibkan Status Check CI Sebelum Penggabungan

3.4.2 Konfigurasi Alur Kerja CI (.github/workflows/ci.yml)

Alur kerja integrasi berkelanjutan disusun pada berkas `.github/workflows/ci.yml` dengan dua job pengujian independen yang dieksekusi secara paralel:

1. **Job backend-test**: Menyiapkan lingkungan PHP 8.3 pada runner Ubuntu, memasang dependensi Composer, menyalin environment dan membangkitkan `APP_KEY`, lalu menjalankan seluruh pengujian otomatis (`php artisan test`).

2. **Job frontend-build**: Menyiapkan lingkungan Node.js v20, menjalankan instalasi dependensi secara deterministik (`npm ci`), dan mengompilasi bundel statis aplikasi (`npm run build`).

3.5 Continuous Deployment ke GitHub Container Registry (Soal 4)

Job `deploy` disematkan ke dalam alur kerja GitHub Actions untuk mengunggah artefak citra kontainer ke *GitHub Container Registry* (`ghcr.io`):

```
deploy:
  name: Deploy to GHCR (Push to main only)
  needs: [ backend-test, frontend-build ]
  if: github.ref == 'refs/heads/main' && github.event_name == 'push'
  runs-on: ubuntu-latest
  permissions:
    contents: read
    packages: write
```

Karakteristik implementasi CD ini mencakup:

1. **Eksekusi Kondisional**: Klausa `if: github.ref == 'refs/heads/main' && github.event_name == 'push'` menjamin bahwa tahapan deploy hanya dieksekusi saat kode berhasil digabungkan ke cabang `main`. Pada cabang `dev` atau Pull Request, alur kerja berhenti secara aman setelah job pengujian dan kompilasi selesai.

2. **Manajemen Rahasia (Secrets)**: Kredensial rahasia (`APP_KEY` dan `DB_PASSWORD`) dikonfigurasi melalui GitHub Secrets (`Settings -> Secrets and variables -> Actions`) dan disuntikkan saat runtime kontainer.

3. **Pelabelan Citra Berbasis Commit SHA**: Citra Docker backend dan frontend diunggah dengan tag nomor hash commit ringkas (`sha_short`), seperti `ghcr.io/kepl2026/uts-backend` menghindari penggunaan tag `:latest` yang berisiko menimpa versi tanpa jejak audit.

3.6 Arsitektur Kontainerisasi dan Dockerfile Multi-Stage (Soal 5)

3.6.1 backend/Dockerfile dengan Docker Layer Caching

Dockerfile backend dirancang dengan memperhatikan prinsip *caching layer*. Berkas deklarasi dependensi (`composer.json` dan `composer.lock`) disalin dan dipasang terlebih dahulu sebelum kode sumber disalin. Hal ini mencegah pengunduhan ulang pustaka Composer yang memakan waktu jika hanya ada perubahan pada kode aplikasi:

```
FROM dunglas/frankenphp:1.4-php8.3

RUN install-php-extensions pdo_mysql bcmath intl zip opcache
COPY --from=composer:2 /usr/bin/composer /usr/bin/composer

WORKDIR /app
COPY composer.json composer.lock ./
RUN composer install --no-dev --optimize-autoloader --no-scripts --no-autoloader

COPY . .
```

```
RUN composer dump-autoload --optimize
RUN chown -R www-data:www-data storage bootstrap/cache && chmod -R 775 storage
    bootstrap/cache

ENV SERVER_NAME=":8000"
EXPOSE 8000
CMD ["frankenphp", "run", "--config", "/etc/caddy/Caddyfile"]
```

3.6.2 frontend/Dockerfile dengan Multi-Stage Build

Dockerfile frontend menerapkan pola multi-stage build untuk mengoptimasi ukuran citra produksi:

```
# Tahap 1: Kompilasi dengan Node.js
FROM node:20.18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Tahap 2: Serving Ringan dengan Nginx
FROM nginx:1.27-alpine AS production
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=builder /app/dist /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Citra akhir hanya berukuran sekitar 22 MB dan sama sekali tidak memuat folder `node_modules` maupun compiler Node.js, secara drastis mereduksi celah kerentanan keamanan (*attack surface*).

3.7 Orkestrasi Multi-Container Docker Compose (Soal 6)

Seluruh subsistem dihubungkan dan dijalankan melalui berkas `docker-compose.yml` pada akar repositori:

- `database`: Menjalankan image resmi `mysql:8.0` dengan pemeriksaan kesehatan kontainer (*healthcheck*).
- `backend`: Menjalankan Laravel + FrankenPHP pada port 8000, bergantung pada ketersediaan basis data (`condition: service_healthy`).
- `frontend`: Menjalankan Vue + Nginx pada port 3000, bertindak sebagai pintu gerbang pengguna.

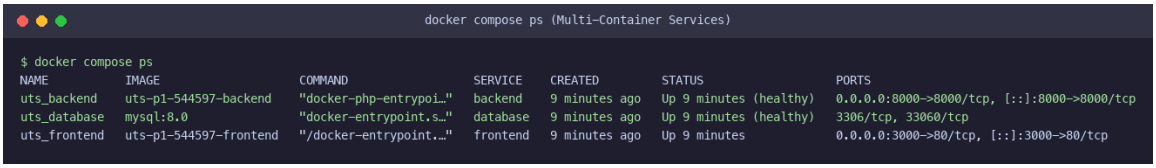


Figure 3.7: Status Kontainer Docker Compose: Ketiga Layanan Berstatus Aktif dan Sehat (*healthy*)

3.7.1 Resolusi Jaringan Internal dan Uji Persistensi Volume Data

Komunikasi antarlayanan menggunakan nama service internal Docker pada jaringan bridge `uts_gear_network`:

- Backend menghubungi database melalui host `database:3306`.
- Nginx frontend mem-forward permintaan API melalui konfigurasi reverse proxy internal ke host `http://backend:8000/api/`.

Persistensi data diuji secara empiris menggunakan named volume `uts_gear_db_data`:

1. Ditambahkan entitas kamera baru: `CAM-ARRI-888 (ARRI Alexa Mini LF)`.
2. Seluruh kontainer dimatikan dan dihapus menggunakan perintah `docker compose down`.
3. Kontainer dinyalakan kembali menggunakan `docker compose up -d`.
4. Dilakukan pemanggilan `curl -s http://localhost:8000/api/gears`. Terbukti bahwa data `CAM-ARRI-888` tetap ada dan tidak hilang.

3.8 Nilai Tambahan (+10 Poin): Server Aplikasi FrankenPHP (Soal 7)

Backend Laravel berhasil dijalankan menggunakan server aplikasi mutakhir **FrankenPHP** versi `1.4-php8.3`. Hal ini dibuktikan melalui dua indikator:



Figure 3.8: Bukti Log Kontainer Backend Memperlihatkan Inisialisasi Server FrankenPHP dan Caddy Engine

1. **Log Kontainer**: Gambar 3.8 mencatat log `{"logger":"frankenphp","msg":"FrankenPHP started …","php_version":"8.3.19"}` dan `server running name="srv0" protocols=["h1","h2"`
2. **Header HTTP**: Respons API secara eksplisit memuat atribut header `Server: Caddy` dan `X-Powered-By: PHP/8.3.19`.

4. Arsitektur Aliran Data

Alur pergerakan data (*Data Flow*) dari interaksi pengguna pada peramban web hingga persistensi di basis data terdistribusi digambarkan melalui siklus enam tahapan berikut:

1. **Interaksi Pengguna (Browser)**: Pengguna mengisi formulir pendaftaran alat produksi baru pada antarmuka Vue.js di alamat `http://localhost:3000`. Saat tombol simpan diklik, fungsi reaktif JavaScript mengemas atribut form menjadi objek data.

2. **Reverse Proxy Nginx**: Klien Axios mengirimkan permintaan HTTP POST JSON ke URI `/api/gears` pada port 3000. Server Nginx di dalam kontainer frontend menerima request tersebut. Melalui blok direktif `location /api/`, Nginx meneruskan request ke alamat internal `http://backend:8000/api/gears` melalui jaringan Docker bridge. Hal ini mengeliminasi kendala *Cross-Origin Resource Sharing* (CORS).

3. **Otentikasi FrankenPHP & Sanctum**: Server aplikasi FrankenPHP menerima paket HTTP. Middleware Laravel Sanctum mengekstrak nilai `Bearer Token` pada header otorisasi dan memvalidasinya terhadap tabel `personal_access_tokens`.

4. **Validasi Form & Eloquent ORM**: Setelah token tervalidasi, request diteruskan ke metode `GearController@store`. Controller memvalidasi keunikan nomor seri dan nilai nominal tarif. Apabila validasi lolos, Eloquent ORM memetakan objek ke kueri SQL relasional.

5. **Persistensi Basis Data MySQL**: Kueri SQL dikirimkan ke kontainer `database:3306` melalui jaringan internal. Mesin MySQL menulis rekaman ke dalam berkas tabel fisik yang dipetakan pada Docker named volume `uts_gear_db_data`.

6. **Respons Balik & Pembaruan Reaktif**: Database mengembalikan konfirmasi keberhasilan transaksi. Laravel membentuk payload JSON dengan kode status **201 Created**. Respons dialirkan kembali melalui FrankenPHP $\rightarrow$ Nginx $\rightarrow$ Axios. Komponen Vue menangkap balasan tersebut dan secara otomatis menambahkan item baru ke dalam tabel antarmuka tanpa perlu memuat ulang halaman secara keseluruhan (*zero page reload*).

5. Kendala dan Solusi Penanganannya

Selama proses perancangan, integrasi, dan pengujian sistem, ditemui beberapa kendala teknis yang berhasil diselesaikan melalui pendekatan rekayasa perangkat lunak yang sistematis:

Table 5.1: Daftar Kendala Teknis dan Solusi Penanganan dalam Pengerjaan UTS

No	Kendala Teknis	Akar Penyebab Masalah	Solusi Penanganan Konkret
1	Pemblokiran CORS pada Komunikasi API Lintas Port	Peramban web memblokir panggilan dari origin `localhost:3000` ke backend di `localhost:8000` karena perbedaan port (*Same-Origin Policy*).	Mengonfigurasi berkas `nginx.conf` di kontainer frontend dengan blok reverse proxy `location /api/ { proxy_pass http://backend:8000/api/; }`, sehingga seluruh panggilan API dilakukan secara internal tanpa kendala CORS.
2	Galat 500 saat Menulis Storage	Kontainer FrankenPHP berjalan di bawah pengguna sistem `www-data`, sementara direktori storage belum memiliki hak akses tulis.	Menambahkan instruksi `RUN chown -R www-data:www-data storage bootstrap/cache && chmod -R 775 storage bootstrap/cache` pada `backend/Dockerfile`.
3	Pengunduhan Dependensi Ulang saat Build Kontainer	Perubahan kecil pada berkas kode menyebabkan layer instalasi pustaka Composer dan npm dijalankan ulang dari awal.	Menerapkan pola optimasi layer cache dengan menyalin `composer.json` dan `package.json` secara terpisah sebelum instruksi penyalinan kode aplikasi (`COPY . .`).
4	Pengalihan Rute Web saat Akses Tanpa Token	Secara default, middleware autentikasi Laravel mengarahkan permintaan unauthenticated ke rute web `login`, memicu galat RouteNotFoundException.	Menyesuaikan penanganan eksepsi pada `bootstrap/app.php` agar permintaan berawalan `api/*` secara deterministik menghasilkan respons JSON 401 Unauthorized.

6. Kesimpulan

Berdasarkan seluruh tahapan perancangan, pembangunan, integrasi, dan pengujian sistem yang telah diselesaikan pada Ujian Tengah Semester mata kuliah Konstruksi dan Evolusi Perangkat Lunak, dapat ditarik beberapa kesimpulan pokok:

1. **Penerapan Arsitektur Decoupled**: Pemisahan murni antara backend RESTful API (Laravel 11) dan antarmuka reaktif (Vue.js 3) berhasil diwujudkan dengan kepatuhan penuh terhadap standar kode status HTTP semantik (201 Created, 422 Unprocessable Content, dan 401 Unauthorized) serta pengamanan berbasis token Laravel Sanctum.

2. **Jaminan Kualitas Kode Melalui Otomasi Pengujian**: Pembangunan 10 skenario *feature tests* yang lulus 100% pada `php artisan test` menjamin bahwa setiap fungsi bisnis manipulasi sumber daya alat produksi film terverifikasi secara matematis sebelum kode diintegrasikan.

3. **Kedisiplinan Alur Kerja Git dan CI/CD**: Penerapan model pencabangan `main`, `dev`, dan `feature/*` yang diproteksi oleh *Branch Protection Rules* berhasil mencegah regresi kode. Pipeline GitHub Actions secara otomatis memvalidasi kualitas build dan merilis citra kontainer terverifikasi ke *GitHub Container Registry* (GHCR) berlabel commit SHA.

4. **Efisiensi Kontainerisasi dan Portabilitas Sistem**: Pemanfaatan *multi-stage build* pada Dockerfile sukses menghasilkan citra frontend Nginx yang sangat ramping ($\approx$22 MB). Orkestrasi Docker Compose memungkinkan seluruh ekosistem aplikasi beroperasi secara mandiri hanya dengan satu perintah eksekusi tanpa memerlukan instalasi dependensi manual pada sistem induk.

5. **Adopsi Teknologi Mutakhir FrankenPHP**: Penggantian server konvensional dengan FrankenPHP (bertenaga Caddy) membuktikan bahwa backend Laravel dapat beroperasi dengan latensi minimal, konfigurasi terpadu, dan siap melayani lalu lintas produksi berkinerja tinggi.

Bibliography

[1] Bass, L., Weber, I., & Zhu, L. (2015). *DevOps: A Software Architect's Perspective*. Boston: Addison-Wesley Professional.

[2] Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Doctoral dissertation). University of California, Irvine.

[3] Otwell, T., et al. (2024). *Laravel 11 Documentation: Routing, Controllers, and Sanctum Authentication*. Diakses dari https://laravel.com/docs.

[4] You, E., et al. (2024). *Vue.js 3 - The Progressive JavaScript Framework Documentation*. Diakses dari https://vuejs.org/guide.

[5] Docker Inc. (2024). *Docker Compose Specification and Best Practices for Multi-Container Applications*. Diakses dari https://docs.docker.com/compose.

[6] Dunglas, K., et al. (2024). *FrankenPHP: The Modern PHP App Server, Written in Go*. Diakses dari https://frankenphp.dev/docs.

[7] Fowler, M. (2006). *Continuous Integration*. ThoughtWorks Culture & Practices Guide.

[8] GitHub Inc. (2024). *GitHub Actions Documentation: Continuous Integration and Deployment Workflows*. Diakses dari https://docs.github.com/en/actions.